ECE479 - Final Project
Improving TimeGAN with Transformer Components for Financial Time-Series Generation

Andrew Yuan

1 Introduction

Modeling financial time-series data is important in quantitative finance, risk management, and algorithmic trading. Market data is sequential, noisy, nonstationary, and affected by changing volatility regimes and cross-feature dependencies, making realistic data generation difficult. Generating realistic synthetic market data can expand limited datasets and support simulation, backtesting, and model evaluation, especially for rare market events that are underrepresented in historical data.

Generative models address this by learning to produce synthetic sequences that resemble real market behavior. TimeGAN is designed for sequential data generation by combining adversarial training with supervised temporal learning, allowing it to preserve both distributional patterns and time-dependent dynamics [1]. In this project, TimeGAN is used as the baseline for generating synthetic stock data, and Transformer-based modifications are explored as alternatives to its original recurrent architecture.

2 Background and Motivations

The original TimeGAN architecture is based on recurrent networks, with its main components implemented using GRUs. These networks are well suited for sequential data because they process time series in order and maintain hidden states across time. However, because recurrent models update their representations step by step, they can have difficulty capturing longer-range dependencies in financial sequences.

This motivates the use of Transformer-based components, which rely on self-attention to compare time steps directly. In financial time-series generation, this may help the model better capture broader temporal patterns such as trends, volatility shifts, and repeated price movements. This project investigates whether replacing selected recurrent components in TimeGAN with Transformer modules improves the quality of generated financial data.

3 Methodology

TimeGAN consists of an *embedder*, *recovery network*, *generator*, *supervisor*, and *discriminator*. The embedder and recovery networks form an autoencoder that maps real sequences into a latent space and reconstructs them, while the generator, supervisor, and discriminator support adversarial training and temporal consistency.

Two modifications were tested: a Transformer embedder and a Transformer discriminator. The Transformer embedder tests whether self-attention improves the learned latent representation, while

the Transformer discriminator tests whether it improves the adversarial signal used to distinguish real and synthetic sequences.

The setup follows the original TensorFlow TimeGAN tutorial pipeline, using the PyTorch version with modifications to selected model components [2, 3]. The dataset consists of daily historical Google stock data from 2004 to 2019, with six features: volume, high price, low price, opening price, closing price, and adjusted closing price [1]. Each sequence contains 24 trading days and 6 stock features.

The Transformer experiments used a hidden dimension of 24, a noise dimension of 6, a batch size of 128, and 4 attention heads. The modified components were the embedder, the discriminator, and the combined embedder-discriminator replacement. To study the effect of model depth and capacity, three Transformer hyperparameter configurations were tested, as shown in Table 1.

Table 1: Transformer hyperparameter configurations tested in TimeGAN.

Config.	d_{model}	**Heads**	**Layers**	**FFN Dim.**
1	24	4	3	96
2	24	4	2	48
3	24	4	2	96

Configuration 1 served as the main Transformer setting, using 3 layers and a feedforward dimension of 96. Configurations 2 and 3 reduced the number of layers to 2 in order to test whether a smaller Transformer could perform similarly or better. Configuration 2 also reduced the feedforward dimension to 48, while Configuration 3 retained the larger feedforward dimension of 96.

4 Evaluation and Benchmarks

Following the same evaluation criteria as in the original TimeGAN paper, the generated sequences were evaluated using three main criteria: diversity, fidelity, and usefulness. Diversity measures whether the synthetic samples cover the real data distribution, while fidelity measures whether the generated samples are difficult to distinguish from real data. Usefulness measures whether the synthetic data preserves meaningful temporal structure for prediction tasks [1].

Three evaluation methods were used. PCA and t-SNE visualizations compare the overlap between real and synthetic samples in low-dimensional space. Ideally, the real and synthetic samples should overlap exactly. The discriminative score measures how well a classifier can distinguish real sequences from generated sequences, with lower scores indicating more realistic synthetic data. The predictive score evaluates whether models trained on synthetic data can predict real data, with lower scores indicating better preservation of temporal dynamics.

5 Results

Among all the configurations and model variants, The Transformer Discriminator variant performed best overall, achieving the lowest discriminative score and predictive score. Compared with the original TimeGAN baseline, it improved the discriminative score from 0.198 to 0.131 and the predictive score from 0.0375 to 0.0360.

Table 2: Results on the stock dataset. Bold indicates best performance.

Metric	Method	Score
Discriminative Score (Lower is better)	Original TimeGAN	0.198 ± 0.008
	Config. 1: TE	0.224 ± 0.041
	Config. 1: TD	$\mathbf{0.131 \pm 0.022}$
	Config. 1: TETD	0.277 ± 0.054
	Config. 2: TETD	0.318 ± 0.084
	Config. 3: TETD	0.258 ± 0.049
Predictive Score (Lower is better)	Original TimeGAN	0.0375 ± 0.0004
	Config. 1: TE	0.0402 ± 0.0003
	Config. 1: TD	$\mathbf{0.0360 \pm 0.0001}$
	Config. 1: TETD	0.0419 ± 0.0006
	Config. 2: TETD	0.0394 ± 0.0001
	Config. 3: TETD	0.0399 ± 0.0010

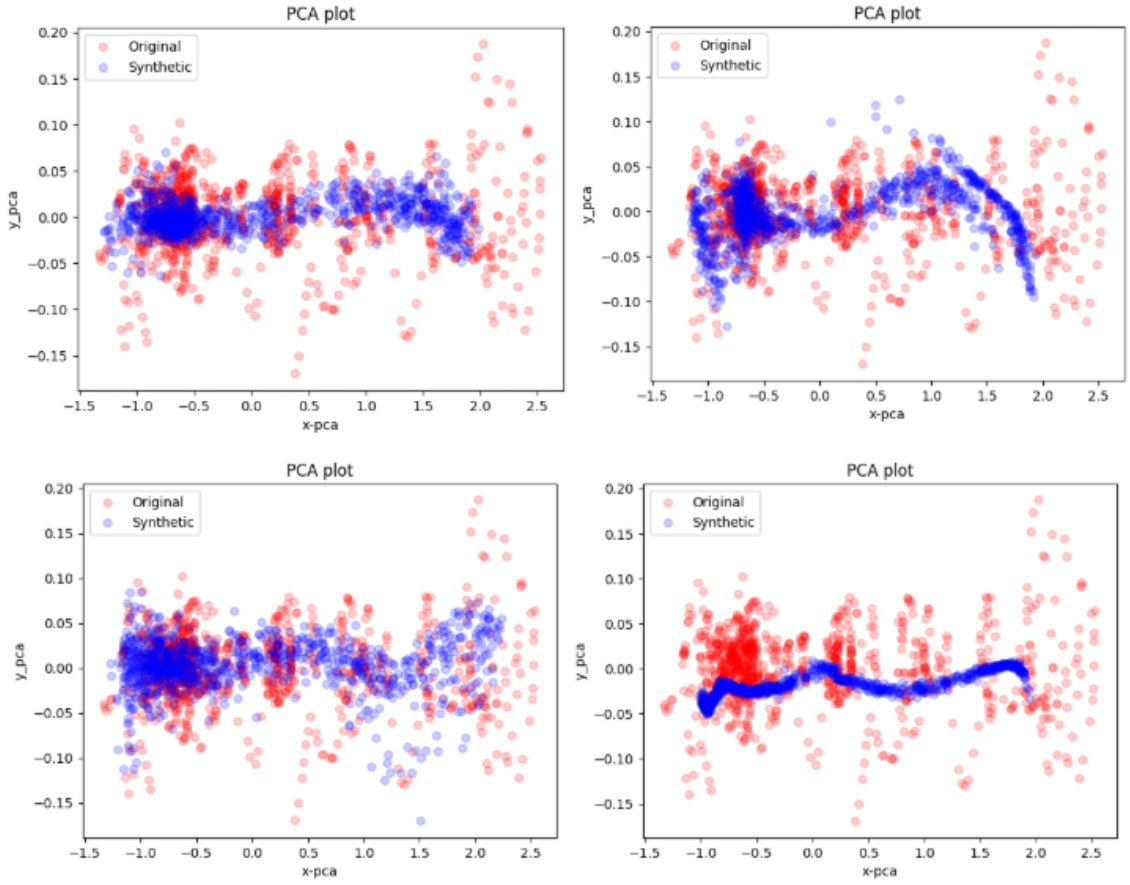

Figure 1: PCA visualizations comparing 1,000 real and 1,000 synthetic stock sequences for each Config. 1 TimeGAN variant: original TimeGAN(left) and TE(right) in the first row, and TD(left) and TETD (right) in the second row.

Replacing only the discriminator with a Transformer improved performance by strengthening the adversarial evaluation of sequence realism while preserving the original TimeGAN generator and latent-space structure. The self-attention mechanism of the Transformer allows direct comparison across time steps, which may improve the detection of temporal inconsistencies that the original GRU-based discriminator could miss. This, in turn, may provide a stronger adversarial signal to the generator, explaining the improved performance.

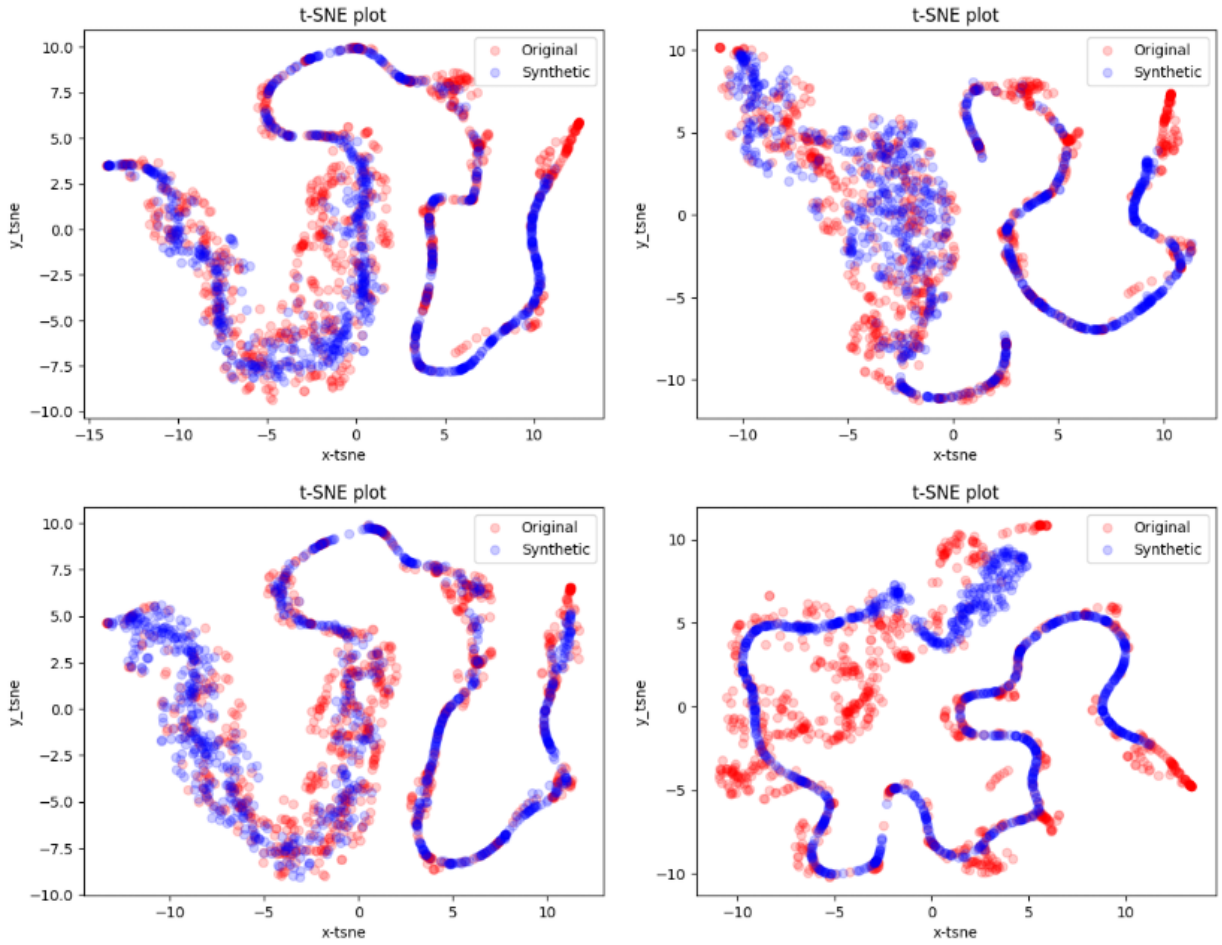

Figure 2: t-SNE visualizations comparing 1,000 real and 1,000 synthetic stock sequences for each Config. 1 TimeGAN variant: original TimeGAN (left) and TE (right) in the first row, and TD (left) and TETD (right) in the second row.

In contrast, replacing the embedder and both the embedder and discriminator did not improve performance. The Transformer embedder worsened the discriminative score, while the combined Transformer variants performed worse than the baseline on both metrics, suggesting that changing the latent representation learned by the embedder made training less stable, especially for relatively short 24-step stock sequences.

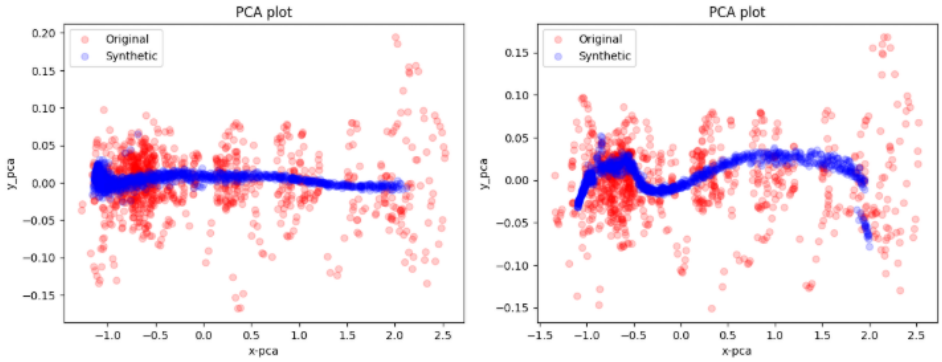

Figure 3: PCA visualizations comparing 1,000 real and 1,000 synthetic stock sequences for Config. 2 TETD (left) and Config. 3 TETD (right).

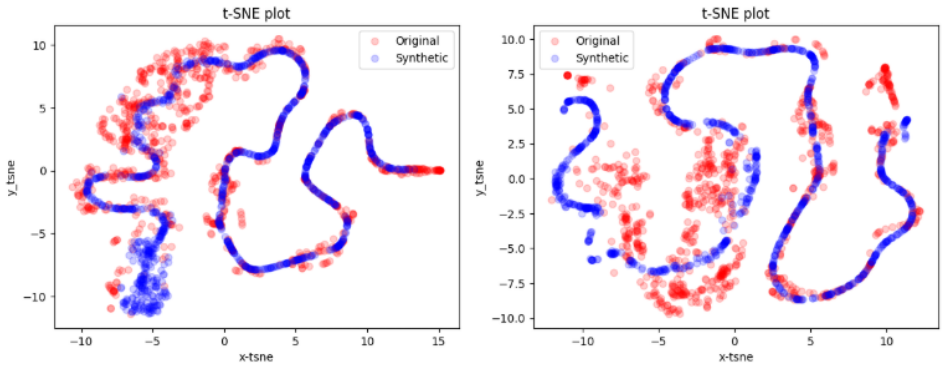

Figure 4: t-SNE visualizations comparing 1,000 real and 1,000 synthetic stock sequences for Config. 2 TETD (left) and Config. 3 TETD (right).

6 Conclusion

This project showed that selectively integrating Transformer components into TimeGAN can improve financial time-series generation. In particular, adding a Transformer discriminator improved TimeGAN's overall performance, suggesting that modern self-attention mechanisms can strengthen older recurrent-based architectures.

Future Work

The Transformer parameters were chosen with limited tuning, so further experimentation is needed to determine whether better configurations could improve performance. Future work should explore a wider range of Transformer depths, feedforward dimensions, attention heads, and training settings. In addition, other components such as the supervisor of the original TimeGAN architecture could be modified to further study how modern models can improve older recurrent-based generative models.

References

[1] J. Yoon, D. Jarrett, and M. van der Schaar, "Time-series generative adversarial networks," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019.

[2] J. Yoon, "TimeGAN: Time-series generative adversarial networks," https://github.com/jsyoon0823/TimeGAN, 2019.

[3] Z. Zhang, "TimeGAN-PyTorch: A pytorch implementation of time-series generative adversarial networks," https://github.com/zwzhang123/TimeGAN-pytorch, 2020.